also avoid excess load being generated on your application servers. In this example you'll apply simple rate-limiting in the API server itself using Google's Guava library. Even if you enforce rate-limiting at a proxy server, it is good security practice to also enforce rate limits in each server so that if the proxy server misbehaves or is misconfigured it is still difficult to bring down the individual servers. This is an instance of the general security principle known as defense in depth, which aims to ensure that no failure of a single mechanism is enough to compromise your API.

> **DEFINITION** The ***principle of defense in depth*** states that multiple layers of security defenses should be used so that a failure in any one layer is not enough to breach the security of the whole system.

As you'll now discover, there are libraries available to make basic rate-limiting very easy to add to your API, while more complex requirements can be met with off-the-shelf proxy/gateway products. Open the pom.xml file in your editor and add the following dependency to the dependencies section:

```
        <dependency>
            <groupId>com.google.guava</groupId>
            <artifactId>guava</artifactId>
            <version>27.0.1-jre</version>
        </dependency>
```

Guava makes it very simple to implement rate-limiting using the `RateLimiter` class that allows us to define the rate of requests per second you want to allow[1]. You can then